

Henry Rotunno

6 May 2026

Natural Disasters, Artificial Models: Which Machine Learning Model is the Best at Classifying Earthquake Threats?

In [2]:

```
import pandas as pd
from sklearn import model_selection, metrics, preprocessing,
neighbors, pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV

df = pd.read_csv('earthquake_alert_balanced_dataset.csv')
df.head(3)
```

Out[2]:

	magnitude	depth	cdi	mmi	sig	alert
0	7.0	14.0	8.0	7.0	0.0	green
1	6.9	25.0	4.0	4.0	-33.0	green
2	7.0	579.0	3.0	3.0	-13.0	green

K Nearest Neighbors

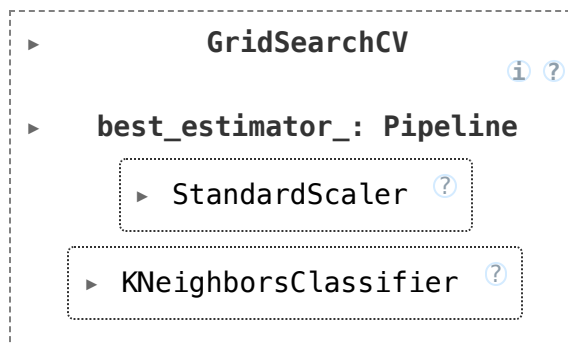
```
In [3]: xval = model_selection.KFold(n_splits = 10, shuffle=True,
    random_state=5)

x = ['magnitude', 'depth', 'cdi', 'mmi', 'sig']

scaler = StandardScaler()

# TODO --> Grid Search for best neighbors parameter
# CHANGE n_neighbors
pipe = pipeline.Pipeline([
    ('scaling', scaler),
    ('model', neighbors.KNeighborsClassifier(n_neighbors=4))
])
grid = {'model__n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]}
grid_se = GridSearchCV(pipe, grid, cv=xval)
grid_se.fit(df[x], df.alert)
```

Out[3]:



```
In [4]: kn_df = pd.DataFrame(grid_se.cv_results_)  
kn_df
```

Out[4]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time
0	0.005754	0.001625	0.009214	0.000988
1	0.004476	0.000931	0.008760	0.001525
2	0.005323	0.001044	0.010141	0.001006
3	0.007461	0.001358	0.014360	0.002380
4	0.006722	0.001363	0.022666	0.016524
5	0.005567	0.001891	0.011457	0.002032
6	0.005413	0.001300	0.014296	0.003882
7	0.008898	0.003478	0.016687	0.002657
8	0.006511	0.002142	0.011854	0.003028
9	0.005940	0.000779	0.014345	0.001760
10	0.004746	0.001053	0.011556	0.002746
11	0.005415	0.001087	0.010887	0.001615

```

In [5]: # I'm going with k=4

scorer = {
    'kappa': metrics.make_scorer(metrics.cohen_kappa_score),
    'precision': metrics.make_scorer(metrics.precision_score,
    average='micro'),
    'recall': metrics.make_scorer(metrics.recall_score,
    average='micro'),
}

results = model_selection.cross_validate(
    pipe,
    df[x],
    df.alert,
    error_score='raise',
    return_train_score=True,
    return_estimator=True,
    scoring=scorer
)

print('train kappa mean:', results['train_kappa'].mean())
print('test kappa mean:', results['test_kappa'].mean())
print('train kappa std:', results['train_kappa'].std())
print('test kappa std: ', results['test_kappa'].std())

print('-----')

print('train precision mean: ', results['train_precision'].mean())
print('test precision: mean ', results['test_precision'].mean())
print('train precision std: ', results['train_precision'].std())
print('test precision std: ', results['test_precision'].std())

print('-----')

print('train recall mean: ', results['train_recall'].mean())
print('test recall: mean ', results['test_recall'].mean())
print('train recall std: ', results['train_recall'].std())
print('test recall std: ', results['test_recall'].std())

```

```

train kappa mean: 0.8733333333333334
test kappa mean: 0.8020512820512821
train kappa std: 0.008480941235149317
test kappa std: 0.03171205094100573
-----
train precision mean: 0.905
test precision: mean 0.8515384615384616
train precision std: 0.0063607059263619875
test precision std: 0.02378403820575432
-----
train recall mean: 0.905

```

```
test recall: mean 0.8515384615384616  
train recall std: 0.0063607059263619875  
test recall std: 0.02378403820575432
```

Logistic Regression

```
In [6]: from sklearn.linear_model import LogisticRegression

pipe_log = pipeline.Pipeline([
    ('scaling', preprocessing.StandardScaler()),
    ('model', LogisticRegression(penalty='l2'))
])

grid_log = {
    'model__C': [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1,
                1.5,
                2.0, 3.0, 3.5, 4.0, 4.5, 6.0, 7.0]
}

grid_search_log = GridSearchCV(
    pipe_log,
    grid_log,
    cv=xval
)

grid_search_log.fit(df[x], df.alert)

pipe_log = pipeline.Pipeline([
    ('scaling', preprocessing.StandardScaler()),
    ('model', LogisticRegression(penalty='l2',
    C=grid_search_log.best_params_['model__C']))
])

results_log = model_selection.cross_validate(
    pipe_log,
    df[x],
    df.alert,
    cv=xval,
    scoring=scorer,
    return_estimator=True,
    return_train_score=True,
)

print('train kappa mean:', results_log['train_kappa'].mean())
print('test kappa mean:', results_log['test_kappa'].mean())
print('train kappa std:', results_log['train_kappa'].std())
print('test kappa std:', results_log['test_kappa'].std())

print('-----')

print('train precision mean: ',
```

```

results_log['train_precision'].mean())
print('test precision: mean ', results_log['test_precision'].mean())
print('train precision std: ', results_log['train_precision'].std())
print('test precision std: ', results_log['test_precision'].std())

print('-----')

print('train recall mean: ', results_log['train_recall'].mean())
print('test recall: mean ', results_log['test_recall'].mean())
print('train recall std: ', results_log['train_recall'].std())
print('test recall std: ', results_log['test_recall'].std())

```

```

train kappa mean: 0.5478444997055492
test kappa mean: 0.5475013875143323
train kappa std: 0.010478513358418151
test kappa std: 0.07370705636912062
-----
train precision mean: 0.660940170940171
test precision: mean 0.6615384615384615
train precision std: 0.007843248442573433
test precision std: 0.05578912112144229
-----
train recall mean: 0.660940170940171
test recall: mean 0.6615384615384615
train recall std: 0.007843248442573433
test recall std: 0.05578912112144229

```

In [7]: `print(grid_search_log.best_params_['model__C'])`

2.0

Random Tree Classifier

```
In [8]: from sklearn import tree

pipe_tree = pipeline.Pipeline([
    ('model', tree.DecisionTreeClassifier(min_samples_leaf=8))
])

results_tree = model_selection.cross_validate(
    pipe_tree,
    df[x],
    df.alert,
    cv=xval,
    scoring=scorer,
    return_train_score=True,
    return_estimator=True,
)

grid_log = {
    'model__min_samples_leaf': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
                                12, 13, 14, 15,
                                20, 25, 30, 35, 40, 45, 50]
}

grid_se_tree = GridSearchCV(
    pipe_tree,
    grid_log,
    cv=xval
)

grid_se_tree.fit(df[x], df.alert)

print('train kappa mean:', results_tree['train_kappa'].mean())
print('test kappa mean:', results_tree['test_kappa'].mean())
print('train kappa std:', results_tree['train_kappa'].std())
print('test kappa std:', results_tree['test_kappa'].std())

print('-----')

print('train precision mean: ',
      results_tree['train_precision'].mean())
print('test precision: mean ',
      results_tree['test_precision'].mean())
print('train precision std: ',
      results_tree['train_precision'].std())
```



```
print('test precision std: ', results_tree['test_precision'].std())

print('-----')

print('train recall mean: ', results_tree['train_recall'].mean())
print('test recall: mean ', results_tree['test_recall'].mean())
print('train recall std: ', results_tree['train_recall'].std())
print('test recall std: ', results_tree['test_recall'].std())
```

```
train kappa mean: 0.836672288268872
test kappa mean: 0.7523301311363768
train kappa std: 0.006187044242356386
test kappa std: 0.044699251277204984
```

```
-----
train precision mean: 0.8775213675213676
test precision: mean 0.8153846153846154
train precision std: 0.004635127132634599
test precision std: 0.03352999187338979
```

```
-----
train recall mean: 0.8775213675213676
test recall: mean 0.8153846153846154
train recall std: 0.004635127132634599
test recall std: 0.03352999187338979
```

Random Forest

```
In [9]: from sklearn.ensemble import RandomForestClassifier

pipe_forest = pipeline.Pipeline([
    ('model', RandomForestClassifier(n_estimators=90, max_depth=10,
min_samples_leaf=1, random_state=30))
])

result_forest = model_selection.cross_validate(
    pipe_forest,
    df[x],
    df.alert,
    cv=xval,
    scoring=scorer,
    return_train_score=True,
    return_estimator=True
)

print('train kappa mean:', result_forest['train_kappa'].mean())
print('test kappa mean:', result_forest['test_kappa'].mean())
print('train kappa std:', result_forest['train_kappa'].std())
print('test kappa std:', result_forest['test_kappa'].std())

print('-----')

print('train precision mean: ',
result_forest['train_precision'].mean())
print('test precision: mean ',
result_forest['test_precision'].mean())
print('train precision std: ',
result_forest['train_precision'].std())
print('test precision std: ', result_forest['test_precision'].std())

print('-----')

print('train recall mean: ', result_forest['train_recall'].mean())
print('test recall: mean ', result_forest['test_recall'].mean())
print('train recall std: ', result_forest['train_recall'].std())
print('test recall std: ', result_forest['test_recall'].std())

print('-----')
```

```
train kappa mean: 0.9667201234184463
test kappa mean: 0.8585568591406808
```

```
train kappa std: 0.004883637431099085
test kappa std: 0.039921807775220405
```

```
train precision mean: 0.975042735042735
test precision: mean 0.8946153846153846
train precision std: 0.0036622709889963206
test precision std: 0.030000000000000002
```

```
train recall mean: 0.975042735042735
test recall: mean 0.8946153846153846
train recall std: 0.0036622709889963206
test recall std: 0.030000000000000002
```

```

In [10]: grid_forest = {
          'model__n_estimators':
          [10,20,30,40,50,60,70,80,90,100,110,120,130,140,150,
           160,170,180,190,200]
        }

        grid_forest = GridSearchCV(
            pipe_forest,
            grid_forest,
            cv=xval
        )

        grid_forest.fit(df[x], df.alert)

        results = pd.DataFrame(grid_forest.cv_results_)
        results

```

Out[10]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time
0	0.021221	0.000975	0.003025	0.000629
1	0.038043	0.001183	0.003625	0.000428
2	0.055808	0.001843	0.004294	0.000439
3	0.083208	0.017281	0.005314	0.000391
4	0.112363	0.012437	0.006759	0.000702
5	0.131924	0.022155	0.008282	0.001405
6	0.143872	0.012307	0.007757	0.000740
7	0.180300	0.052427	0.009386	0.002589
8	0.173202	0.006425	0.009303	0.000911
9	0.184423	0.006198	0.009771	0.000930
10	0.200439	0.006575	0.010231	0.000340

	mean_fit_time	std_fit_time	mean_score_time	std_score_time
11	0.221651	0.009226	0.011012	0.000946
12	0.245612	0.005727	0.012090	0.000795
13	0.259610	0.010218	0.012791	0.001057
14	0.268435	0.005681	0.012828	0.000402
15	0.319686	0.054712	0.015031	0.002104
16	0.401980	0.070333	0.019072	0.004885
17	0.375199	0.051550	0.016758	0.002132
18	0.433265	0.076291	0.024262	0.018924
19	0.594090	0.200034	0.020409	0.003914

```

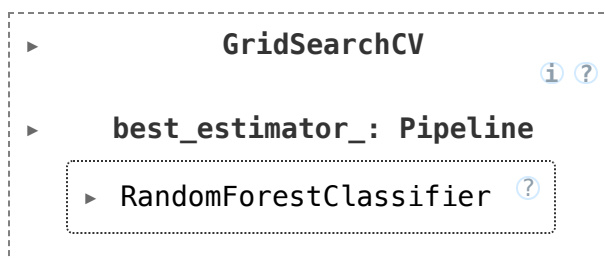
In [11]: grid_forest_depth = {
        'model__max_depth': [None, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15,
                               17, 20, 22, 25, 30]
        }

grid_forest_depth = GridSearchCV(
    pipe_forest,
    grid_forest_depth,
    cv=xval
)

grid_forest_depth.fit(df[x], df.alert)

```

Out[11]:



```
In [12]: df_r = pd.DataFrame(grid_forest_depth.cv_results_)  
df_r
```

Out[12]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time
0	0.197156	0.034247	0.010107	0.002200
1	0.150585	0.042373	0.009181	0.001706
2	0.143093	0.028436	0.010788	0.006160
3	0.200315	0.035372	0.012072	0.003697
4	0.145440	0.015022	0.008344	0.000821
5	0.169279	0.019048	0.009242	0.001445
6	0.156532	0.009898	0.008643	0.000896
7	0.185491	0.023294	0.010495	0.002648
8	0.199810	0.058568	0.008848	0.001131
9	0.191692	0.014189	0.009839	0.001570
10	0.220493	0.049893	0.013467	0.005602
11	0.222713	0.038256	0.010851	0.001505
12	0.246453	0.062040	0.013949	0.005326
13	0.227098	0.074302	0.009889	0.001709
14	0.185130	0.019140	0.009954	0.001561
15	0.194874	0.028766	0.010088	0.002019

	mean_fit_time	std_fit_time	mean_score_time	std_score_time
16	0.249525	0.067638	0.010845	0.000903

```
In [13]: print(grid_forest_depth.best_params_)  
  
{'model__max_depth': 20}
```

```
In [14]: min_samp_grid = {  
        'model__min_samples_leaf':  
        [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20]  
        }  
  
        grid_forest_samp = GridSearchCV(  
            pipe_forest,  
            min_samp_grid,  
            cv=xval  
        )  
  
        grid_forest_samp.fit(df[x], df.alert)
```

```
Out[14]: 

GridSearchCV



best_estimator_: Pipeline



RandomForestClassifier


```

```
In [15]: df_grid_for_samp = pd.DataFrame(grid_forest_samp.cv_results_)
df_grid_for_samp
```

Out[15]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time
0	0.194486	0.037097	0.010510	0.001967
1	0.219612	0.046380	0.011149	0.002288
2	0.195235	0.034919	0.009994	0.001167
3	0.224592	0.072423	0.010992	0.004421
4	0.158418	0.011264	0.008966	0.000982
5	0.230439	0.079864	0.011590	0.003482
6	0.146521	0.005841	0.008007	0.000219
7	0.144427	0.003972	0.008308	0.000457
8	0.149382	0.013243	0.008128	0.000672
9	0.142708	0.003307	0.008557	0.000823
10	0.164663	0.057639	0.009461	0.003854
11	0.146123	0.003176	0.008515	0.000419
12	0.146294	0.005789	0.008619	0.000676
13	0.145760	0.005281	0.008301	0.000597
14	0.150508	0.004392	0.008664	0.000664
15	0.150460	0.007549	0.009120	0.000785

	mean_fit_time	std_fit_time	mean_score_time	std_score_time
16	0.192836	0.043963	0.010694	0.002824
17	0.167678	0.017423	0.009766	0.001182
18	0.182356	0.018080	0.011334	0.002027
19	0.162979	0.019069	0.009571	0.001360

Multi Layer Perceptron: Neural Network

```
In [16]: from sklearn import neural_network

pipe_perc = pipeline.Pipeline([
    ('model', neural_network.MLPClassifier(hidden_layer_sizes=
(200,100), max_iter=400))]
])

result_mlp = model_selection.cross_validate(
    pipe_perc,
    df[x],
    df.alert,
    cv=xval,
    scoring=scorer,
    return_train_score=True,
    return_estimator=True,
    error_score='raise'
)
```

```

In [17]: print('train kappa mean:', result_mlp['train_kappa'].mean())
print('test kappa mean:', result_mlp['test_kappa'].mean())
print('train kappa std:', result_mlp['train_kappa'].std())
print('test kappa std:', result_mlp['test_kappa'].std())

print('-----')

print('train precision mean: ',
      result_mlp['train_precision'].mean())
print('test precision: mean ', result_mlp['test_precision'].mean())
print('train precision std: ', result_mlp['train_precision'].std())
print('test precision std: ', result_mlp['test_precision'].std())

print('-----')

print('train recall mean: ', result_mlp['train_recall'].mean())
print('test recall: mean ', result_mlp['test_recall'].mean())
print('train recall std: ', result_mlp['train_recall'].std())
print('test recall std: ', result_mlp['test_recall'].std())

```

```

train kappa mean: 0.733871361558625
test kappa mean: 0.6856107833346026
train kappa std: 0.050943630686756336
test kappa std: 0.05660027409083637

```

```

-----
train precision mean: 0.8003418803418804
test precision: mean 0.7661538461538461
train precision std: 0.03824096294533692
test precision std: 0.042856579314412675

```

```

-----
train recall mean: 0.8003418803418804
test recall: mean 0.7661538461538461
train recall std: 0.03824096294533692
test recall std: 0.042856579314412675

```

```
In
[18]: mlp_grid = {
      'model__hidden_layer_sizes': [
          (25,),
          (50,),
          (100,),
          (100, 50),
          (100, 100),
          (200, 100),
          (200, 200),
          (250, 200)
      ]
    }
    mlp_grid_search = GridSearchCV(
        pipe_perc,
        mlp_grid,
        cv=xval
    )

    mlp_grid_search.fit(df[x], df.alert)
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/pytho
n3.12/site-
```

```
packages/sklearn/neural_network/_multilayer_perceptron.py:69
```

```
1: ConvergenceWarning: Stochastic Optimizer: Maximum
iterations (400) reached and the optimization hasn't
converged yet.
```

```
warnings.warn(
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/pytho
n3.12/site-
```

```
packages/sklearn/neural_network/_multilayer_perceptron.py:69
```

```
1: ConvergenceWarning: Stochastic Optimizer: Maximum
iterations (400) reached and the optimization hasn't
converged yet.
```

```
warnings.warn(
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/pytho
n3.12/site-
```

```
packages/sklearn/neural_network/_multilayer_perceptron.py:69
```

```
1: ConvergenceWarning: Stochastic Optimizer: Maximum
iterations (400) reached and the optimization hasn't
converged yet.
```

```
warnings.warn(
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/pytho
n3.12/site-
```

```
packages/sklearn/neural_network/_multilayer_perceptron.py:69
```

```
1: ConvergenceWarning: Stochastic Optimizer: Maximum
iterations (400) reached and the optimization hasn't
converged yet.
```

```
warnings.warn(
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/pytho
n3.12/site-
```

```
packages/sklearn/neural_network/_multilayer_perceptron.py:69
```

```
1: ConvergenceWarning: Stochastic Optimizer: Maximum
```



```
n3.12/site-  
packages/sklearn/neural_network/_multilayer_perceptron.py:69  
1: ConvergenceWarning: Stochastic Optimizer: Maximum  
iterations (400) reached and the optimization hasn't  
converged yet.  
  warnings.warn(
```

Out[18]:

GridSearchCV

best_estimator_: Pipeline

MLPClassifier

In [19]:

df_mlp = pd.DataFrame(mlp_grid_search.cv_results_)
df_mlp

Out[19]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time
0	0.723899	0.079462	0.002491	0.000674
1	0.977172	0.170947	0.002514	0.000877
2	0.756367	0.201591	0.002087	0.000270
3	1.654622	0.470529	0.003475	0.002008
4	1.418163	0.371055	0.002477	0.000505
5	1.378273	0.549900	0.002700	0.000648
6	2.274930	0.732085	0.004009	0.002818
7	2.485261	0.964919	0.003068	0.000731

```
In [20]: import matplotlib.pyplot as plt
import seaborn as sns

colors = [
    '#6556b0',
    '#baa843',
    '#12db51',
    '#0e3d05',
    '#14a2b8']

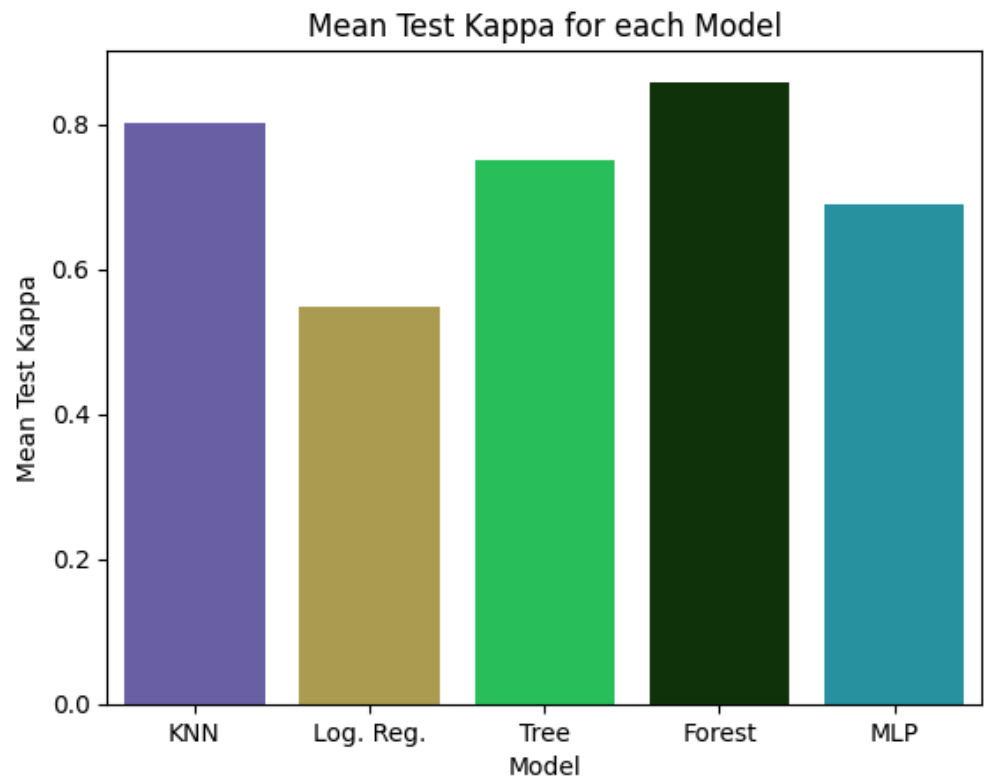
data = {'KNN': 0.8020512820512821,
        'Log. Reg.': 0.5475013875143323,
        'Tree': 0.7513057178385288,
        'Forest': 0.8585568591406808,
        'MLP': 0.6897073038376884}

ax = sns.barplot(x=list(data.keys()), y=list(data.values()),
                 palette=colors)
ax.set(title="Mean Test Kappa for each Model", xlabel="Model",
       ylabel="Mean Test Kappa")
plt.show()
```

```
/var/folders/n8/0phyq9hj1dg695h802qmqr080000gn/T/ipykernel_2
1624/3784836580.py:19: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=list(data.keys()),
                 y=list(data.values()), palette=colors)
```



In []:

In []:

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).